ENTREGA #2 — FAST PROMPTING EN ACCIÓN

Feria del Automóvil: generación inteligente de leads con Prompt Engineering

Desentrañando la magia

ALUMNA
María Luján Massironi

CURSO
Prompt Engineering

COMISIÓN
N.º 96165

ENTREGABLE
POC en Jupyter Notebook

1. Introducción

Nombre del proyecto

Feria del Automóvil: Generación Inteligente de Leads con Prompt Engineering

Presentación del problema

La problemática seleccionada es la pérdida de potenciales clientes dentro del proceso comercial del sector automotriz.

Actualmente, una gran parte del proceso de búsqueda de un automóvil comienza de manera digital. Los usuarios comparan vehículos, consultan alternativas de financiación y buscan información antes de comunicarse directamente con un vendedor.

Esta problemática también puede presentarse en ferias o exposiciones automotrices, donde muchas personas observan vehículos y muestran interés, pero no necesariamente dejan sus datos o continúan posteriormente el contacto con la empresa.

El desafío consiste entonces en encontrar una manera simple de:

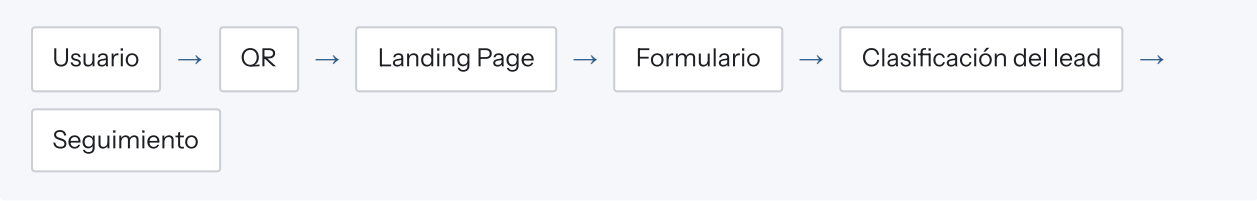
- captar el interés del usuario;
- obtener información relevante;
- identificar su intención de compra;

- generar contenido adaptado a sus necesidades;
- facilitar el posterior contacto comercial.

El proyecto propone utilizar **Prompt Engineering e Inteligencia Artificial Generativa** como herramientas de apoyo para este proceso.

2. Propuesta de solución

La solución consiste en desarrollar una POC (*Proof of Concept*) que demuestre cómo diferentes técnicas de **Fast Prompting** permiten generar contenido más preciso y útil para la captación de leads. El proceso general planteado es:



La POC se concentrará principalmente en dos tareas:

Generación de contenido

A partir de información básica del contexto automotriz, el modelo generará contenido para una landing page destinada a captar potenciales compradores.

Seguimiento personalizado

A partir de algunos datos de un lead — nombre, vehículo de interés, forma de pago y plazo estimado de compra— el modelo generará un mensaje comercial personalizado.

La notebook permitirá modificar estos datos para demostrar que un mismo prompt estructurado puede reutilizarse en diferentes situaciones.

3. Objetivos

Objetivo general

Desarrollar una POC mediante una Jupyter Notebook que permita demostrar cómo las técnicas de Fast Prompting pueden mejorar la generación de contenido para la captación y el seguimiento de potenciales clientes del sector automotriz.

Objetivos específicos

- Diseñar prompts claros y reutilizables.

- Comparar un prompt básico con un prompt optimizado.
- Aplicar técnicas de Zero-Shot Prompting.
- Incorporar Role Prompting para proporcionar contexto al modelo.
- Utilizar restricciones para controlar las respuestas.
- Definir formatos de salida específicos.
- Incorporar Few-Shot Prompting cuando sea necesario.
- Generar contenido personalizado utilizando variables.
- Reducir la cantidad de consultas innecesarias a la API.
- Evaluar si las técnicas utilizadas mejoran la solución planteada en la primera entrega.

4. Metodología

El proyecto se desarrollará mediante una metodología experimental basada en la creación, prueba y optimización progresiva de prompts. Primero se utilizará un prompt básico con pocas instrucciones. Por ejemplo:

```
Escribe un mensaje para vender un auto.
```

Este prompt puede generar una respuesta correcta, pero presenta un problema: el modelo no conoce el contexto, el público, el objetivo comercial, la extensión esperada ni el tono.

Posteriormente se aplicarán técnicas de Fast Prompting para agregar información específica. La estructura utilizada será:

`ROL + CONTEXTO + TAREA + DATOS + RESTRICCIONES + FORMATO DE SALIDA`

> EJEMPLO DE PROMPT OPTIMIZADO
>
> Actúa como asesor comercial especializado en venta de automóviles.
>
> Un potencial cliente llamado Juan está interesado en un Toyota Corolla, desea financiar la compra y piensa adquirir el vehículo dentro de los próximos tres meses.
>
> Redacta un mensaje de seguimiento. Requisitos:
>
> - máximo 100 palabras;
> - tono profesional y cercano;
> - no inventar precios;
> - no inventar condiciones de financiación;
> - incluir una única llamada a la acción.
>
> Devuelve únicamente el mensaje destinado al cliente.

La comparación permitirá observar cómo el segundo prompt reduce la ambigüedad y genera una respuesta más adecuada para el problema planteado.

5. Técnicas de Fast Prompting

Zero-Shot Prompting

Se utilizará cuando la tarea pueda explicarse directamente mediante instrucciones sin proporcionar ejemplos anteriores —por ejemplo, generar el contenido de una landing page. Esta técnica permite mantener prompts relativamente breves y reducir el consumo innecesario de tokens.

Role Prompting

Se asignará al modelo un rol relacionado con la tarea. Ejemplo:

> Actúa como copywriter especializado en marketing automotriz.

El rol proporciona contexto sobre el tipo de respuesta esperada.

Prompt con contexto

Se incluirá solamente la información necesaria para resolver cada tarea. En lugar de solicitar simplemente:

> Escribe un mensaje para un cliente.

se proporcionarán datos como vehículo de interés, forma de pago y plazo estimado de compra.

Uso de restricciones

Los prompts incluirán restricciones explícitas para evitar respuestas demasiado extensas o información inventada. Por ejemplo:

- máximo 100 palabras;
- no inventar precios;
- no inventar descuentos;
- utilizar una única llamada a la acción;
- mantener un tono profesional pero cercano.

Formato de salida

También se especificará cómo debe responder el modelo. Esto permite obtener resultados más consistentes y facilita una posible integración futura con otras aplicaciones.

Few-Shot Prompting

Para tareas donde se necesite mayor consistencia se podrá proporcionar uno o dos ejemplos del resultado esperado. Sin embargo, esta técnica se utilizará solamente cuando sea necesaria, ya que agregar demasiados ejemplos aumenta la cantidad de tokens utilizados y, por lo tanto, el costo de las consultas.

6. Implementación de la POC

La notebook comenzará instalando la librería necesaria e importando las dependencias:

```
!pip install openai

from openai import OpenAI
import os
```

La API Key no se escribirá directamente dentro del código publicado en GitHub:

```
client = OpenAI(
    api_key=os.getenv("OPENAI_API_KEY")
)
```

De esta manera se evita publicar información privada dentro del repositorio.

7. Primera prueba: prompt básico

```
prompt_basico = """
Escribe un mensaje para un cliente interesado en comprar un auto.
"""

print(prompt_basico)
```

Este prompt presenta varias limitaciones porque no indica:

- qué automóvil busca el cliente;
- cuál es su intención de compra;
- qué tono utilizar;
- qué extensión debe tener;
- qué objetivo tiene el mensaje;
- qué información puede o no inventar el modelo.

Por este motivo se desarrollará una segunda versión.

8. Prompt optimizado con Fast Prompting

```
nombre = "Juan"
vehiculo = "Toyota Corolla"
forma_pago = "financiación"
plazo_compra = "1 a 3 meses"

prompt_optimizado = f"""
ROL:
Actúa como asesor comercial especializado en venta de automóviles.

CONTEXTO:
Un potencial cliente completó un formulario de interés.

DATOS DEL LEAD:
Nombre: {nombre}
Vehículo de interés: {vehiculo}
Forma de pago: {forma_pago}
Plazo estimado de compra: {plazo_compra}

TAREA:
Redacta un mensaje de seguimiento para continuar el contacto comercial.

RESTRICCIONES:
- Máximo 100 palabras.
- Utiliza un tono profesional pero cercano.
- No inventes precios.
- No inventes descuentos.
- No inventes condiciones de financiación.
- No presiones al cliente para realizar la compra.
- Incluye una única llamada a la acción.

FORMATO:
Devuelve únicamente el mensaje destinado al cliente.
"""

print(prompt_optimizado)
```

Esta segunda versión utiliza **Role Prompting + contexto + variables + restricciones + formato de salida**.

9. Consulta al modelo

Para evitar realizar consultas innecesarias durante las pruebas, se ejecutará solamente el prompt optimizado.

```
response = client.responses.create(
    model="gpt-5.6",
    input=prompt_optimizado
)

resultado = response.output_text

print(resultado)
```

El resultado esperado es un mensaje personalizado que utiliza los datos proporcionados sin inventar información comercial.

10. Función reutilizable

Para mejorar la POC, el código puede transformarse en una función.

```
def generar_seguimiento(nombre, vehiculo, forma_pago, plazo_compra):

    prompt = f"""
    ROL:
    Actúa como asesor comercial especializado en venta de automóviles.

    CONTEXTO:
    Un potencial cliente completó un formulario de interés.

    DATOS:
    Nombre: {nombre}
    Vehículo: {vehiculo}
    Forma de pago: {forma_pago}
    Plazo de compra: {plazo_compra}

    TAREA:
    Redacta un mensaje de seguimiento comercial personalizado.

    RESTRICCIONES:
    - Máximo 100 palabras.
    - Tono profesional y cercano.
    - No inventar precios.
    - No inventar descuentos.
    - No inventar condiciones de financiación.
    - Una única llamada a la acción.

    FORMATO:
    Devuelve solamente el mensaje para el cliente.
    """

    response = client.responses.create(
        model="gpt-5.6",
        input=prompt
    )

    return response.output_text
```

Luego puede reutilizarse:

```
mensaje = generar_seguimiento(
    "María",
    "Volkswagen Taos",
    "financiación",
    "1 a 3 meses"
)

print(mensaje)
```

Esto permite utilizar una única estructura de prompt para diferentes leads.

11. Interactividad

Como contenido adicional, la notebook puede permitir que la persona ingrese los datos.

```
nombre = input("Nombre del cliente: ")
vehiculo = input("Vehículo de interés: ")
forma_pago = input("Forma de pago: ")
plazo_compra = input("Plazo estimado de compra: ")

mensaje = generar_seguimiento(
    nombre,
    vehiculo,
    forma_pago,
    plazo_compra
)

print("\nMENSAJE GENERADO:\n")
print(mensaje)
```

De esta forma la POC deja de utilizar solamente información cargada manualmente en el código y permite realizar una demostración interactiva.

12. Segunda aplicación: landing page

La misma metodología puede aplicarse a la generación del contenido de una landing page.

```
prompt_landing = """
ROL:
Actúa como copywriter especializado en marketing automotriz.

CONTEXTO:
La Feria del Automóvil necesita una landing page accesible mediante
un código QR para captar personas interesadas en comprar vehículos.

OBJETIVO:
Lograr que el visitante complete un formulario de contacto.

TAREA:
Genera el contenido principal de la landing page.

RESTRICCIONES:
- Headline de máximo 10 palabras.
- Subheadline de máximo 20 palabras.
- Tres beneficios.
- Un único CTA.
- Tono profesional y cercano.
- No inventar descuentos, precios ni promociones.

FORMATO:
HEADLINE:
SUBHEADLINE:
BENEFICIOS:
1.
2.
3.
CTA:
"""

response = client.responses.create(
    model="gpt-5.6",
    input=prompt_landing
)

print(response.output_text)
```

13. Modelo texto → imagen

El proyecto también contempla la generación de imágenes a partir de prompts. Para esta etapa se utilizará un prompt estructurado que defina **sujeto + acción + contexto + estilo + composición**.

```
prompt_imagen = """
Fotografía publicitaria realista de una feria automotriz moderna.

Sujeto:
Un potencial comprador joven utilizando un smartphone.

Acción:
Está escaneando un código QR ubicado junto a un automóvil.

Contexto:
Stand de una concesionaria dentro de una exposición automotriz moderna.

Estilo:
Fotografía publicitaria corporativa y realista.

Composición:
Automóvil visible en primer plano, visitante escaneando el QR,
iluminación profesional, fondo de exposición ligeramente desenfocado.

Evitar:
Texto ilegible, marcas inventadas, vehículos deformados,
elementos futuristas exagerados.
"""

print(prompt_imagen)
```

Este prompt puede copiarse y probarse posteriormente en una herramienta de generación de imágenes disponible.

La POC principal utilizará la API para las tareas de texto y mantendrá la generación visual como una demostración complementaria, evitando generar imágenes mediante API en cada ejecución y reduciendo así consultas y costos innecesarios.

14. Optimización de consultas y costos

Uno de los objetivos de la implementación es evitar llamadas innecesarias a la API. Por este motivo se aplicarán las siguientes decisiones:

1. Los prompts pueden visualizarse y modificarse sin consultar la API.
2. La API solamente se ejecuta cuando el usuario solicita generar el resultado.
3. No se generan múltiples respuestas cuando una sola es suficiente.
4. Se limita la extensión solicitada en cada prompt.
5. Se utiliza Zero-Shot Prompting cuando no son necesarios ejemplos.

6. Few-Shot se reserva para situaciones donde realmente mejore la consistencia.
7. Las imágenes no se generan automáticamente cada vez que se ejecuta la notebook.

De esta manera, la POC demuestra la solución sin realizar consultas innecesarias.

15. Comparación entre prompt básico y Fast Prompting

CARACTERÍSTICA	PROMPT BÁSICO	PROMPT OPTIMIZADO
Rol	No	Sí
Contexto	Mínimo	Específico
Datos del cliente	No	Sí
Restricciones	No	Sí
Formato esperado	No	Sí
Reutilizable	Poco	Sí
Control de información inventada	Bajo	Mayor
Personalización	Baja	Alta

La aplicación de Fast Prompting permite transformar una instrucción general en una estructura más controlada y reutilizable.

16. Análisis de la mejora respecto de la Preentrega 1

La primera entrega permitió definir la problemática, establecer el alcance del proyecto y diseñar los primeros prompts. En esta segunda etapa, la propuesta evoluciona desde un planteo conceptual hacia una **Proof of Concept funcional**. Las técnicas aprendidas permiten mejorar principalmente tres aspectos.

Mayor precisión

La incorporación de roles, contexto, restricciones y formatos de salida reduce la ambigüedad del prompt.

Mayor reutilización

El uso de variables permite mantener una misma estructura y modificar únicamente los datos de cada potencial cliente.

Mayor eficiencia

Definir el resultado esperado desde la primera consulta reduce la necesidad de solicitudes correctivas.

Esto resulta especialmente importante cuando se utilizan APIs cuyo costo depende de la cantidad de consultas y tokens procesados.

17. Conclusión

La POC demuestra que el Prompt Engineering puede aplicarse a una problemática concreta del sector automotriz sin necesidad de desarrollar o entrenar un modelo de Inteligencia Artificial desde cero.

A través de técnicas de Fast Prompting es posible controlar mejor el contexto, las restricciones y el formato de las respuestas generadas. En comparación con los prompts iniciales, las nuevas estructuras permiten obtener contenido más específico, reutilizable y alineado con el objetivo comercial.

Además, la implementación mediante Python demuestra que estos prompts pueden dejar de ser instrucciones aisladas y convertirse en componentes reutilizables dentro de una aplicación.

La solución mantiene un alcance técnicamente viable, ya que requiere pocas consultas a la API y puede demostrarse completamente mediante una Jupyter Notebook.